

ACTIVIDAD PRÁCTICA — SEMANA 7

Consultas SQL y pandas · Desarrollo profesional

Caso: empresa de distribución de alimentos perecederos. Se trabaja con información de clientes, pedidos, rutas y productos, siguiendo el modelo planteado en la Semana 6. La guía solicita tres consultas SQL —WHERE, JOIN y GROUP BY—, reproducir la agregación en pandas y explicar cada consulta.



Flujo de consulta → integración → agregación → análisis

1. Dataset utilizado

Para hacer las consultas reproducibles se plantea un conjunto relacional coherente con el ERD del caso anterior. Las tablas principales son CLIENTE, PEDIDO, RUTA y PRODUCTO. Para la consulta de productos por pedido se utiliza también DETALLE_PEDIDO.

Ejemplo de registros relevantes:

CLIENTE	PEDIDO	RUTA	PRODUCTO
101 · Supermercado Norte	5001 · 2026-09-20 · Entregado	R01 · Neiva → Bogotá	201 · Leche entera
102 · Tienda Central	5002 · 2026-09-21 · Pendiente	R02 · Neiva → Pitalito	202 · Yogur natural
103 · Mercado Sur	5003 · 2026-09-22 · Entregado	R03 · Neiva → Garzón	203 · Queso fresco

2. Consulta SQL con WHERE

Objetivo: identificar los pedidos que cumplen una condición específica. En este caso se filtran los pedidos que tienen estado 'Pendiente'.

```
SELECT id_pedido, id_cliente, id_ruta, fecha, estado FROM PEDIDO WHERE estado = 'Pendiente';
```

¿Qué responde? Esta consulta responde qué pedidos están pendientes de atención. El uso de WHERE limita el conjunto de registros sin modificar los datos originales. Es útil para monitorear órdenes que requieren seguimiento logístico.

3. Consulta SQL con JOIN

Objetivo: combinar información de dos entidades relacionadas. Se unen PEDIDO y CLIENTE mediante la FK id_cliente para conocer quién realizó cada pedido.

```
SELECT p.id_pedido, c.nombre AS cliente, p.fecha, p.estado FROM PEDIDO p INNER JOIN CLIENTE c ON  
p.id_cliente = c.id_cliente;
```

¿Qué responde? Permite consultar los pedidos junto con el nombre del cliente. El JOIN evita repetir los datos descriptivos del cliente dentro de cada registro de pedido y utiliza la relación establecida mediante PK/FK.

4. Consulta SQL con GROUP BY

Objetivo: obtener una medida agregada por cliente. Se calcula la cantidad de pedidos registrados para cada cliente.

```
SELECT c.id_cliente, c.nombre AS cliente, COUNT(p.id_pedido) AS total_pedidos FROM CLIENTE c LEFT JOIN  
PEDIDO p ON c.id_cliente = p.id_cliente GROUP BY c.id_cliente, c.nombre ORDER BY total_pedidos DESC;
```

¿Qué responde? Muestra cuántos pedidos tiene cada cliente. Se utiliza COUNT para contar pedidos y GROUP BY para agruparlos por cliente. El LEFT JOIN permite conservar también clientes que todavía no tengan pedidos registrados.

5. Reproducción de GROUP BY en pandas

La consulta agregada puede reproducirse en pandas utilizando un DataFrame de pedidos y otro de clientes. Primero se realiza el merge equivalente al JOIN y después el groupby equivalente al GROUP BY.

```
import pandas as pd
clientes = pd.DataFrame({ "id_cliente": [101, 102, 103], "nombre": ["Supermercado Norte", "Tienda Central", "Mercado Sur"] })
pedidos = pd.DataFrame({ "id_pedido": [5001, 5002, 5003, 5004], "id_cliente": [101, 101, 102, 103], "estado": ["Entregado", "Pendiente", "Entregado", "Entregado"] })
resultado = ( clientes.merge(pedidos, on="id_cliente", how="left") .groupby(["id_cliente", "nombre"], as_index=False) .agg(total_pedidos=("id_pedido", "count")) .sort_values("total_pedidos", ascending=False) )
print(resultado)
```

Equivalencia conceptual: merge() realiza la integración de tablas, mientras groupby() organiza los registros por las columnas seleccionadas y agg() aplica la función de agregación. Por tanto, pandas reproduce la lógica analítica de la consulta SQL dentro de Python.

6. Resultado esperado

id_cliente	cliente	total_pedidos
101	Supermercado Norte	2
102	Tienda Central	1
103	Mercado Sur	1

7. Interpretación de las tres consultas

La consulta WHERE sirve para localizar registros que cumplen una condición operativa; en este caso, pedidos pendientes. La consulta JOIN integra información relacionada entre tablas y permite visualizar el pedido junto con su cliente. Finalmente, GROUP BY transforma registros individuales en un indicador agregado, como el número de pedidos por cliente.

Estas tres operaciones representan una progresión habitual en análisis de datos: filtrar → integrar → resumir. La misma lógica puede trasladarse a pandas para continuar el análisis, construir indicadores y posteriormente generar visualizaciones o reportes.

8. Conclusión

El ejercicio demuestra cómo SQL y pandas pueden utilizarse de forma complementaria. SQL resulta adecuado para consultar datos estructurados y relacionados, mientras pandas permite reproducir las operaciones y continuar el análisis en Python. El resultado es un flujo reproducible que convierte registros operativos en información útil para la toma de decisiones logísticas.

9. Estructura sugerida para GitHub

La guía indica que la entrega corresponde a la carpeta 07-week/ del repositorio. Una estructura ordenada puede ser:

```
07-week/ ■■■ README.md ■■■ consultas.sql ■■■ analisis_pandas.py ■■■ datos/ ■■■ clientes.csv ■■■ pedidos.csv
```

Referencias de apoyo

Documentación oficial de SQL y pandas como material técnico complementario. Para la entrega, conservar la estructura y requisitos indicados por la guía de Semana 7.